

NER wolof avec un CRF

Un repère rapide entre le notebook, les fichiers Python et les idées à défendre.

La phrase qui résume le projet

J'ai construit un système de reconnaissance d'entités nommées en wolof. Le CRF prédit une séquence de labels BIO à partir de la forme des mots, de leur voisinage et des transitions entre labels.

Type	Signification	Exemple
PER	Personne	Maki Sàll
LOC	Lieu	Dakar
ORG	Organisation	ONU
DATE	Date	12 mars 2024

Résultat honnête

F1 dev sélectionné : 0,7583

F1 test strict : 0,6622

La baisse sur test montre que le modèle généralise moins bien sur les entités rares. Ce n'est pas un échec caché : c'est une limite mesurée et expliquée.

Application : <https://5aaxfh5jf9kkjgjna7fpso.streamlit.app/>

1. La carte complète du projet

Suis toujours les flèches. Elles représentent l'ordre réel d'exécution.

Étape	Question simple	Preuve à ouvrir
1. Données	Que contient le corpus ?	Notebook 5-11 ; analyze_data.py
2. Préparation	Comment le texte devient X/y ?	Notebook 12-13 ; features.py ; dataset.py
3. Apprentissage	Que mémorise le CRF ?	Notebook 14-17 ; train_crf.py
4. Dev	Comment mesure-t-on et choisit-on ?	Notebook 18-28 ; evaluate.py ; select_crf.py
5. Test	Le modèle généralise-t-il ?	Notebook 29-31 ; finalize_crf.py
6. Démo	Comment prédire une nouvelle phrase ?	Notebook 32-33 ; predict.py ; app.py

Le protocole en une ligne

train apprend les poids → dev choisit c1/c2 → test mesure une seule fois le modèle figé.

Arborescence à montrer

Code à reconnaître

data/raw/ corpus train, dev, test
src/ logique Python réutilisable
models/ modèle final + résultats
projet_nlp.ipynb démonstration progressive
app.py interface Streamlit

Si le jury demande : « Où est le raisonnement ? »

Réponse : « Le notebook raconte et prouve chaque étape. Le dossier src contient les mêmes opérations sous forme de fonctions courtes, réutilisables et testables. »

2. Données CoNLL et labels BIO

Notebook : cellules 5 à 11 | Fichier : src/analyze_data.py

Une phrase CoNLL est un bloc de lignes « token label ». Une ligne vide sépare deux phrases.

Code à reconnaître

Maki B-PER
Sàll I-PER
dem O
Dakar B-LOC

Label	Interprétation	Règle
B-TYPE	Début d'une entité	ouvre une nouvelle entité
I-TYPE	Continuation	doit suivre B/I du même type
O	Hors entité	aucune entité à cette position

Fonctions essentielles

Fonction	Ce qu'elle garantit
load_conll(path)	lit les phrases, refuse les lignes et labels invalides
find_bio_errors(...)	repère une continuation I-TYPE impossible
count_entities(...)	compte les entités à partir des B-TYPE
print_report(...)	affiche phrases, tokens, labels, entités et erreurs

Pourquoi l'accuracy ne suffit pas ?

Le label O est majoritaire. Un modèle qui prédit souvent O peut avoir une bonne accuracy tout en ratant les entités. C'est pourquoi le projet utilise le F1 au niveau des entités.

Phrase de soutenance

« Avant d'entraîner, je vérifie la qualité structurelle du corpus : labels autorisés, alignement token-label et transitions BIO valides. »

3. Caractéristiques, X et y

Notebook : cellules 12-13 | Fichiers : src/features.py et src/dataset.py

Principe

Un CRF ne reçoit pas directement une phrase brute. Chaque token devient un petit dictionnaire d'indices observables. X contient ces dictionnaires ; y contient les labels BIO.

Caractéristique	Exemple sur « Sénégal »	Intérêt
word.lower	sénégal	normaliser la casse
prefix/suffix	sé / gal	repérer des formes fréquentes
word.shape	Xxxxxxx	résumer majuscules/minuscules/chiffres
istitle / isupper / isdigit	True / False / False	indices de nom, sigle ou date
mot précédent / suivant	contexte local	désambiguïser le token
BOS / EOS	début / fin	position dans la phrase

Code à reconnaître

```
X_train, y_train = prepare_split(train_sentences)
# Pour chaque phrase i : len(X_train[i]) == len(y_train[i])
```

Lecture ligne par ligne

sentence_features(tokens) appelle token_features pour chaque position. prepare_split construit une liste de séquences X et copie les labels dans y. Aucun vrai label n'entre dans les caractéristiques : il n'y a pas de fuite de réponse.

Question piège : pourquoi le voisinage ?

Un même mot peut changer de rôle selon son contexte. Le mot précédent et le mot suivant donnent au CRF un contexte local sans rendre le modèle complexe.

4. Entraînement du CRF

Notebook : cellules 14-17 | Fichier : src/train_crf.py

Le CRF cherche la meilleure séquence complète de labels, pas une décision isolée pour chaque mot.

Composant	Question qu'il pose	Exemple
Émission	Ce token ressemble-t-il à ce label ?	majuscule → PER possible
Transition	Ce label suit-il bien le précédent ?	B-PER → I-PER
Régularisation	Les poids sont-ils trop spécialisés ?	c1 et c2 limitent le surapprentissage

Code à reconnaître

```
CRF(algorithm="lbfgs", c1=0.1, c2=0.1,  
max_iterations=100,  
all_possible_transitions=True)  
model.fit(X_train, y_train)
```

Paramètre	Explication défendable
lbfgs	algorithme d'optimisation adapté à ce modèle linéaire
c1	régularisation L1 ; peut rendre certains poids nuls
c2	régularisation L2 ; réduit les poids trop grands
100 itérations	limite le temps d'optimisation
toutes transitions	le modèle peut apprendre aussi les transitions rares

Ce que fit() apprend

fit() apprend les poids des caractéristiques et les poids des transitions uniquement à partir de train. Il ne voit ni dev ni test.

5. Évaluation stricte sur dev

Notebook : cellules 18-22 | Fichier : src/evaluate.py

Une entité n'est correcte que si ses limites et son type sont exacts.

Cas	Vrai	Prédit	Diagnostic
Exact	Dakar / LOC	Dakar / LOC	TP
Mauvais type	ONU / ORG	ONU / LOC	FP + FN
Manquée	Dakar / LOC	rien	FN
Inventée	rien	Dakar / LOC	FP
Mauvaises limites	Maki Sàll / PER	Sàll / PER	FP + FN

Métrique	Formule	Question simple
Précision	$TP / (TP + FP)$	Mes prédictions sont-elles fiables ?
Rappel	$TP / (TP + FN)$	Ai-je retrouvé les entités attendues ?
F1	$2PR / (P + R)$	Quel est l'équilibre global ?

Code à reconnaître

```
options = {"mode": "strict", "scheme": IOB2,
"zero_division": 0}
f1_score(y_true, y_pred, **options)
```

Pourquoi segeval ?

segeval reconstruit les entités à partir des séquences BIO. Le mode strict vérifie ensemble le début, la fin et le type de chaque entité.

6. Comprendre les erreurs

Notebook : cellules 23-25 | Fichier : src/error_analysis.py

L'objectif n'est pas seulement de compter les erreurs, mais de savoir pourquoi le modèle échoue.

Catégorie	Condition	Ce que j'en conclus
invented	prédiction sans entité vraie	le modèle sur-détecte
missed	entité vraie sans prédiction	le rappel diminue
wrong_type	mêmes limites, type différent	confusion sémantique
wrong_boundaries	chevauchement, limites différentes	mauvaise segmentation

Comment le code travaille

1. `extract_entities` reconstruit les segments B-TYPE + I-TYPE.
2. Les triplets (début, fin, type) exacts sont retirés.
3. Les prédictions restantes sont comparées aux vraies entités.
4. `spans_overlap` distingue mauvais type, mauvaises limites, invention et oubli.

Bonne attitude scientifique

Je n'affirme pas une cause uniquement à partir d'un score. J'inspecte des exemples réels de dev, puis je relie les erreurs aux données rares, aux mots inconnus ou aux limites ambiguës.

Question du jury : « Pourquoi ne pas corriger test ? »

Parce que test sert uniquement à mesurer la généralisation finale. Toute correction choisie après observation de test contaminerait l'évaluation.

7. Sélection du modèle sans tricher

Notebook : cellules 26-31 | Fichiers : select_crf.py et finalize_crf.py

Quatre configurations seulement sont comparées. C'est limité, lisible et reproductible.

Nom	c1	c2	Rôle
initiale	0,10	0,10	point de départ
L1 forte	0,50	0,10	plus de sélection de poids
L2 forte	0,10	0,50	poids davantage réduits
équilibrée	0,05	0,20	meilleur F1 dev : 0,7583

Décision finale

La configuration c1=0,05 et c2=0,20 maximise le F1 strict sur dev. Elle est ensuite figée avant l'ouverture de test.

Test final	Précision	Rappel	F1
Micro global	0,7749	0,5781	0,6622
LOC	0,8859	0,7725	0,8253
PER	0,7055	0,5852	0,6398
ORG	0,6129	0,3455	0,4419
DATE	0,5238	0,1571	0,2418

Interprétation

LOC généralise bien. DATE et ORG ont surtout un rappel faible : beaucoup d'entités attendues ne sont pas retrouvées. Cela explique le F1 test inférieur au F1 dev.

8. Baseline : prouver l'utilité du CRF

Notebook : cellules 34-36 | Fichier : src/baseline.py

La baseline mémorise, pour chaque mot de train, son label le plus fréquent. Un mot inconnu reçoit O.

Modèle sur dev	Précision	Rappel	F1 strict
Mémorisation	0,8289	0,5780	0,6811
CRF sélectionné	0,7843	0,7339	0,7583

Ce que montre la comparaison

La baseline est précise sur les mots déjà vus, mais elle manque beaucoup d'entités. Le CRF améliore le rappel grâce à la forme des mots, au contexte local et aux transitions BIO.

Lecture du code

Code à reconnaître

```
counts[token.lower()][label] += 1
word_to_label[word] = label_le_plus_frequent
mot_inconnu -> O
```

Question du jury : « Pourquoi une baseline ? »

« Sans point de comparaison, un score isolé ne prouve pas que le CRF apporte quelque chose. La baseline répond à la question : fait-il mieux qu'une simple mémorisation ? »

9. Prédiction et modèle sauvegardé

Notebook : cellules 29-33 | Fichiers : finalize_crf.py et predict.py

Fonction	Entrée	Sortie
tokenize_text	texte libre	mots, nombres et ponctuation
sentence_features	liste de tokens	caractéristiques par token
predict_single	une séquence X	une séquence de labels BIO
extract_entities	tokens + labels	texte, type, début, fin
predict_text	phrase complète	tokens, labels et entités

Code à reconnaître

```
demo = predict_text("Maki Sàll dem Dakar")
print(demo["tokens"])
print(demo["labels"])
print(demo["entities"])
```

Pourquoi deux fichiers dans models/ ?

crf_wolof.joblib contient l'objet CRF appris. crf_wolof_metadata.json garde la configuration et les métriques finales. Le premier sert à prédire ; le second sert à documenter et reproduire.

Chaîne complète à réciter

Texte → tokenisation → caractéristiques → CRF → labels BIO → entités reconstruites.

10. Application Streamlit et mode assisté

Fichiers : app.py et src/postprocess.py

L'application est une interface de démonstration. Elle charge le modèle une seule fois avec cache, prédit les labels puis colore les entités.

Bloc de app.py	Rôle
load_model()	charge models/crf_wolof.joblib une seule fois
predict(...)	tokenise, construit X et lance le CRF
highlighted_text(...)	associe une couleur à PER/LOC/ORG/DATE
widgets Streamlit	saisie, mode assisté, bouton et tableau BIO

Mode assisté

Règle	Exemple	Type proposé
ville sénégalaise connue	Dakar	LOC
sigle en majuscules	ONU	ORG
nombre ou mois	12 mars 2024	DATE
deux mots Title Case	Fatou Ndiaye	PER

Distinction importante

Le score scientifique vient du CRF brut évalué dans le notebook. Le mode assisté est un bonus explicable pour la démonstration ; ses règles ne doivent pas être confondues avec les performances du CRF.

Lien : <https://5aaxfh5jf9kkjgjna7fpso.streamlit.app/>

11. Index express des fichiers Python

Fichier	Une phrase pour l'expliquer	Cellules
analyze_data.py	lit et contrôle le corpus CoNLL/BIO	5-11
features.py	décrit chaque token avec des indices simples	12-13
dataset.py	transforme les phrases en X et y alignés	12-13
train_crf.py	construit le CRF et exécute fit()	14-19
evaluate.py	calcule précision, rappel et F1 strict	20-22
error_analysis.py	classe les vraies erreurs d'entités	23-25
select_crf.py	compare quatre couples c1/c2 sur dev	26-28
finalize_crf.py	fige, teste une fois et sauvegarde	29-31
predict.py	prédit sur une phrase libre	32-33
baseline.py	fournit un point de comparaison simple	34-36
postprocess.py	ajoute les règles du mode assisté	application
app.py	présente le modèle dans Streamlit	démonstration

Ordre conseillé pendant la soutenance

Commence par le notebook. Ouvre un fichier Python seulement si le jury demande le détail d'une fonction.
Termine par l'application, puis reviens sur les limites et le F1 test.

12. Questions probables du jury

Pourquoi un CRF ?

Parce qu'il modélise une séquence, apprend les transitions BIO et reste interprétable avec peu de données.

Pourquoi pas seulement l'accuracy ?

Parce que O est majoritaire ; le F1 strict mesure les entités complètes.

Pourquoi dev et test ?

Dev choisit le modèle. Test mesure une seule fois sa généralisation.

Que font c1 et c2 ?

Ils régularisent les poids et limitent le surapprentissage.

Pourquoi le F1 test baisse ?

DATE et ORG ont un rappel faible et les données test contiennent des cas moins bien couverts.

Le mode assisté améliore-t-il le score ?

Il améliore la démo sur certains motifs, mais il n'entre pas dans le score scientifique du CRF brut.

Quelle amélioration ensuite ?

Comparer à un transformeur préentraîné en gardant le même protocole et la même métrique stricte.

13. Fiche de secours - 90 secondes

1. Problème

Reconnaître PER, LOC, ORG et DATE dans du texte wolof.

2. Données

Corpus CoNLL, labels BIO, séparation train/dev/test.

3. Méthode

Caractéristiques de forme et de voisinage → CRF → séquence BIO.

4. Validation

F1 strict avec sequeval ; sélection de c1/c2 sur dev seulement.

5. Résultats

Baseline dev 0,6811 ; CRF dev 0,7583 ; test final 0,6622.

6. Conclusion

Le CRF est simple et interprétable. LOC généralise bien ; DATE et ORG restent difficiles. L'application permet de tester le modèle et montre clairement ses limites.

Phrase finale

« Mon projet ne cherche pas seulement un score : il montre un protocole propre, une évaluation stricte, une analyse honnête des erreurs et une démonstration utilisable. »